

PYTHON COLLECTIONS

List, Tuple, Set and Dictionary

Complete beginner-friendly notes with definitions, differences, real-life examples, important methods, runnable programs and outputs.

Collection	Real-life meaning
List []	Shopping cart - ordered and changeable
Tuple ()	GPS coordinates - fixed values
Set {}	Unique registrations - no duplicates
Dictionary {key: value}	Employee record - labeled information

Easy rule: Choose the collection based on whether order, duplicates, modification and labels are required.

1. Quick Comparison

Python collections are used to store multiple values in a single variable. Each collection behaves differently.

Type	Syntax	Ordered	Duplicates	Changeable	Access
List	[]	Yes	Yes	Yes	Index
Tuple	()	Yes	Yes	No	Index
Set	{ }	No	No	Yes	No index
Dictionary	{key: value}	Yes*	Keys: No	Yes	Key

* Dictionaries preserve insertion order in modern Python. Their values are accessed mainly by keys.

How to choose?

Requirement	Best choice
Order matters and values can change	List
Values must remain fixed	Tuple
Only unique values are needed	Set
Every value needs a name or label	Dictionary

Index: Python starts counting positions from 0. The first item is at index 0, the second item is at index 1, and so on.

2. List []

A **list** stores multiple items in order. It allows duplicate values and can be modified after creation.

Real example: A shopping cart. Products remain in an order, the same product can be added more than once, and items can be added or removed.

```
shopping_cart = ["Milk", "Bread", "Rice", "Milk"]
```

```
print(shopping_cart)
print(shopping_cart[0])
```

OUTPUT

```
['Milk', 'Bread', 'Rice', 'Milk']
Milk
```

Common List Methods

Method	Purpose	Example
append(x)	Adds one item at the end	fruits.append("Orange")
insert(i, x)	Adds an item at index i	fruits.insert(1, "Grapes")
extend(items)	Adds multiple items	fruits.extend(["Guava", "Papaya"])
remove(x)	Removes the first matching value	fruits.remove("Banana")
pop(i)	Removes and returns item at index	fruits.pop(1)
index(x)	Returns position of a value	fruits.index("Mango")
count(x)	Counts matching values	numbers.count(10)
sort()	Sorts the original list	numbers.sort()
reverse()	Reverses the original order	fruits.reverse()
clear()	Removes all items	fruits.clear()
copy()	Creates a shallow copy	new_list = fruits.copy()

List Methods - Runnable Example

```
fruits = ["Apple", "Banana", "Mango"]
```

```
fruits.append("Orange")  
print("After append:", fruits)
```

```
fruits.insert(1, "Grapes")  
print("After insert:", fruits)
```

```
fruits.extend(["Guava", "Papaya"])  
print("After extend:", fruits)
```

```
fruits.remove("Banana")  
print("After remove:", fruits)
```

```
removed = fruits.pop(1)  
print("Removed item:", removed)  
print("Final list:", fruits)
```

OUTPUT

```
After append: ['Apple', 'Banana', 'Mango', 'Orange']  
After insert: ['Apple', 'Grapes', 'Banana', 'Mango', 'Orange']  
After extend: ['Apple', 'Grapes', 'Banana', 'Mango', 'Orange', 'Guava', 'Papaya']  
After remove: ['Apple', 'Grapes', 'Mango', 'Orange', 'Guava', 'Papaya']  
Removed item: Grapes  
Final list: ['Apple', 'Mango', 'Orange', 'Guava', 'Papaya']
```

Sorting and Counting

```
numbers = [40, 10, 30, 20, 10]
```

```
print("Count of 10:", numbers.count(10))  
numbers.sort()  
print("Ascending:", numbers)  
numbers.sort(reverse=True)  
print("Descending:", numbers)
```

OUTPUT

```
Count of 10: 2  
Ascending: [10, 10, 20, 30, 40]  
Descending: [40, 30, 20, 10, 10]
```

Important: `remove(value)` removes by value. `pop(index)` removes by position and returns the removed item.

3. Tuple ()

A **tuple** stores ordered values, allows duplicates, but cannot be modified after creation. This is called **immutable**.

Real example: GPS coordinates. Latitude and longitude represent a fixed location and normally should not change accidentally.

```
location = (12.9716, 77.5946)
```

```
print("Latitude:", location[0])
print("Longitude:", location[1])
```

OUTPUT

```
Latitude: 12.9716
Longitude: 77.5946
```

Tuple Methods

Method	Purpose	Example
count(x)	Counts how many times a value appears	marks.count(90)
index(x)	Returns the index of the first matching value	marks.index(75)

```
marks = (80, 90, 75, 90, 85)
```

```
print("90 appears:", marks.count(90), "times")
print("Index of 75:", marks.index(75))
```

OUTPUT

```
90 appears: 2 times
Index of 75: 2
```

Tuple Packing and Unpacking

```
employee = (101, "Ravi", "Developer")
id, name, role = employee
```

```
print(id)
print(name)
print(role)
```

OUTPUT

```
101
Ravi
Developer
```

Not allowed: `marks.append(95)` and `marks.remove(80)`. A tuple has no methods that change its contents.

4. Set { }

A **set** stores unique values. Duplicate values are automatically removed. A set is unordered, so items do not have index positions.

Real example: Unique student registrations. If Ravi registers twice, his name should appear only once.

```
students = {"Ravi", "Anu", "Kiran", "Ravi", "Anu"}  
print(students)
```

OUTPUT

```
{'Ravi', 'Anu', 'Kiran'} # Order may be different
```

Common Set Methods

Method	Purpose	Example
add(x)	Adds one item	<code>courses.add("React")</code>
update(items)	Adds multiple items	<code>courses.update(["AWS", "Azure"])</code>
remove(x)	Removes item; error if missing	<code>courses.remove("Java")</code>
discard(x)	Removes item safely	<code>courses.discard("C++")</code>
pop()	Removes an arbitrary item	<code>courses.pop()</code>
union(other)	All unique items from both sets	<code>a.union(b)</code>
intersection(other)	Items common to both sets	<code>a.intersection(b)</code>
difference(other)	Items only in the first set	<code>a.difference(b)</code>
clear()	Removes all items	<code>courses.clear()</code>

Set Operations - Runnable Example

```
batch1 = {"Python", "Java", "SQL"}
batch2 = {"Java", "React", "SQL"}

print("Union:", batch1.union(batch2))
print("Intersection:", batch1.intersection(batch2))
print("Only in batch1:", batch1.difference(batch2))
```

OUTPUT

```
Union: {'Python', 'Java', 'SQL', 'React'}
Intersection: {'Java', 'SQL'}
Only in batch1: {'Python'}
```

Add and Remove

```
courses = {"Python", "Java", "Angular"}

courses.add("React")
courses.update(["AWS", "Azure"])
courses.remove("Java")
courses.discard("C++")

print(courses)
```

OUTPUT

```
{'Python', 'Angular', 'React', 'AWS', 'Azure'}
# Order may be different
```

remove() vs discard()

Method	If value exists	If value is missing
remove(value)	Removes it	Raises KeyError
discard(value)	Removes it	No error

Empty set: Use `empty_set = set()`. Writing `{}` creates an empty dictionary, not an empty set.

5. Dictionary {key: value}

A **dictionary** stores data as key-value pairs. A key is a label used to access its related value. Keys must be unique, but values may repeat.

Real example: An employee record. Labels such as name, role and city make every value easy to understand.

```
employee = {
    "id": 101,
    "name": "Thaneshwara",
    "role": "Technical Trainer",
    "city": "Bangalore"
}
```

```
print(employee["name"])
print(employee["role"])
```

OUTPUT

```
Thaneshwara
Technical Trainer
```

Common Dictionary Methods

Method	Purpose	Example
get(key)	Reads a value safely	student.get("name")
keys()	Returns all keys	student.keys()
values()	Returns all values	student.values()
items()	Returns key-value pairs	student.items()
update(data)	Adds or changes entries	student.update({"age": 23})
pop(key)	Removes a specified key	student.pop("age")
popitem()	Removes the last inserted pair	student.popitem()
setdefault(k,v)	Adds key only when missing	student.setdefault("city", "Bengaluru")
clear()	Removes all entries	student.clear()
copy()	Creates a shallow copy	new_student = student.copy()

Dictionary Methods - Runnable Example

```
student = {
    "name": "Ravi",
    "age": 22,
    "course": "Python"
}

print("Name:", student.get("name"))
print("City:", student.get("city", "Not available"))
print("Keys:", student.keys())
print("Values:", student.values())

student.update({"age": 23, "city": "Bangalore"})
removed = student.pop("course")

print("Removed:", removed)
print("Updated:", student)
```

OUTPUT

```
Name: Ravi
City: Not available
Keys: dict_keys(['name', 'age', 'course'])
Values: dict_values(['Ravi', 22, 'Python'])
Removed: Python
Updated: {'name': 'Ravi', 'age': 23, 'city': 'Bangalore'}
```

Loop Through a Dictionary

```
employee = {"id": 101, "name": "Ravi", "role": "Developer"}

for key, value in employee.items():
    print(key, ":", value)
```

OUTPUT

```
id : 101
name : Ravi
role : Developer
```

Safe access: `student["city"]` raises `KeyError` if city is missing. `student.get("city", "Not available")` returns the default value safely.

6. Similar Methods - Important Differences

Methods	Difference
<code>list.append(x)</code> vs <code>list.extend(items)</code>	<code>append</code> adds one object; <code>extend</code> adds each item from another collection.
<code>list.remove(x)</code> vs <code>list.pop(i)</code>	<code>remove</code> uses a value; <code>pop</code> uses an index and returns the removed item.
<code>set.remove(x)</code> vs <code>set.discard(x)</code>	<code>remove</code> gives an error when missing; <code>discard</code> does not.
<code>dict["key"]</code> vs <code>dict.get("key")</code>	Square brackets may raise <code>KeyError</code> ; <code>get</code> reads safely and can return a default.
<code>list.sort()</code> vs <code>sorted(list)</code>	<code>sort</code> changes the original list; <code>sorted</code> returns a new sorted list.
list vs tuple	List is changeable; tuple is fixed.

One Combined Real-Time Example

```
# List: ordered products; duplicates allowed
products = ["Laptop", "Mouse", "Keyboard", "Mouse"]

# Tuple: fixed shop location
shop_location = (12.9716, 77.5946)

# Set: unique categories
categories = {"Electronics", "Accessories", "Electronics"}

# Dictionary: labeled product details
laptop = {
    "id": 101,
    "name": "Laptop",
    "price": 55000,
    "available": True
}

print("Products:", products)
print("Location:", shop_location)
print("Categories:", categories)
print("Laptop:", laptop["name"], "-", laptop["price"])
```

OUTPUT

```
Products: ['Laptop', 'Mouse', 'Keyboard', 'Mouse']
Location: (12.9716, 77.5946)
Categories: {'Electronics', 'Accessories'}
Laptop: Laptop - 55000
```

7. Final Revision Sheet

Collection	Remember	Most important methods
List	Ordered, duplicates, changeable	append, insert, extend, remove, pop, sort
Tuple	Ordered, duplicates, fixed	count, index
Set	Unordered, unique, changeable	add, update, discard, union, intersection
Dictionary	Key-value pairs, unique keys	get, keys, values, items, update, pop

Memory Trick

LIST = Shopping cart

TUPLE = Fixed GPS location

SET = Unique registrations

DICTIONARY = Employee or product record

Practice Questions

1. Create a list of five cities and add one more city.
2. Create a tuple containing a student's roll number, name and course.
3. Remove duplicate numbers by converting a list into a set.
4. Create a dictionary containing a product's name, price and stock status.
5. Find common subjects between two student sets using intersection().
6. Loop through an employee dictionary and print every key and value.

Best learning method: Copy each program into VS Code, save it with a .py extension, run it, and then change the values to observe the result.